

# KOTULO

## Technical Concepts & Presentation Guide

---

SAFEX white maize forecasting with SARIMAX, XGBoost and Seq2Seq, combined into one SELL / HOLD recommendation for farmers.

| #  | Contents                 |
|----|--------------------------|
| 1  | The Big Picture          |
| 2  | Data Concepts            |
| 3  | Evaluation Metrics       |
| 4  | SARIMAX (Model 1 of 3)   |
| 5  | XGBoost (Model 2 of 3)   |
| 6  | Seq2Seq (Model 3 of 3)   |
| 7  | The Ensemble             |
| 8  | Presentation Cheat Sheet |
| 9  | Key Talking Points       |
| 10 | Quick Glossary           |

## PART 1

# The Big Picture

## What Kotulo actually does

Kotulo takes three types of historical data:

- **SAFEX white maize prices** (R/ton)
- **External drivers** — ZAR/USD exchange rate, rainfall, fuel price
- **Dates** (business days, holidays, seasonal patterns)

It feeds this data through three different machine learning models, each of which produces a forecast. It then weights those forecasts by how historically accurate each model has been, and produces one single **SELL / HOLD** recommendation a farmer can act on.

### WHERE TO SHOW THIS

Home tab > the paragraph under “Welcome to Kotulo”. This is the 30-second elevator pitch.

## PART 2

# Data Concepts

## 2.1 Time Series Data

### WHAT IT IS

Data where every row is tagged with a date and the order matters. Yesterday affects today. Randomly shuffling rows would destroy the pattern.

### WHY IT MATTERS HERE

Commodity prices are a classic time series. If you shuffle the rows and then split into train/test, the model “sees the future” during training — which is cheating.

### WHERE TO SHOW THIS

Home tab > “Show the data” expander. You’ll see a line chart where each point is a specific business day.

## 2.2 Chronological Train / Test Split

### WHAT IT IS

Instead of randomly splitting data, you split it by time. The first 80% of days train the model. The most recent 20% test it.

### WHY IT MATTERS

This mimics real life — when you make a forecast, you only have the past. Never the future.

### WHERE TO SHOW THIS

Home tab > Advanced settings > “Train / test split” slider. Explain: “80/20 means the model learns from the earliest 80% of days and is tested on the most recent 20%.”

## 2.3 Look-Ahead Bias / Data Leakage

### WHAT IT IS

The sin of accidentally letting the model see the future during training. Classic ways to commit this:

- Fitting a scaler on the full dataset instead of just training data
- Using random K-fold cross-validation on time series
- Using a driver variable that is only known after the forecast date

### WHY IT MATTERS

Every model in Kotulo is guarded against this. Scalers are fit on training data only. Cross-validation uses **TimeSeriesSplit**. Driver features are all lagged by at least one day.

#### WHERE TO SHOW THIS

Get Help tab > “Explaining the Advanced Settings” > “Train / test split”.

## 2.4 Feature Engineering & Lagged Features

### WHAT IT IS

Creating new input columns from the raw data. For example, *Close (t-1)* is yesterday's price, *dClose (t-3)* is the price change three days ago, and *Volatility5* is the rolling standard deviation of price over 5 days.

### WHY IT MATTERS

Machine learning models can only learn from the columns you give them. Well-designed features carry most of the predictive signal.

#### WHERE TO SHOW THIS

Forecast tab > XGBoost > Settings > “Price lags” and “Driver lags”. These control how many historical days the model sees.

## 2.5 Forward-Fill & Resampling

### WHAT IT IS

When drivers arrive at a slower frequency (monthly fuel prices) than the target (daily maize prices), you forward-fill the driver so every day has a value.

### WHY IT MATTERS

Without this, an inner join would keep only ~12 dates per year and throw away 95% of the price data. The app would still run, but with almost nothing to learn from.

#### WHERE TO SHOW THIS

You don't need to show it — but if asked “how do you handle missing driver data?”, this is the answer.

## PART 3

# Evaluation Metrics

## 3.1 MAE — Mean Absolute Error

$$\text{MAE} = \text{mean}(|\text{actual} - \text{predicted}|)$$

**In plain English:** On average, how many rand per ton was the model wrong by? **Why it's used:** Same units as the target (R/ton), so it's easy to interpret. "MAE of 45" means the model is on average 45 rand off.

### WHERE TO SHOW THIS

Forecast tab > run any model > the metrics table shows MAE (R/ton).

## 3.2 RMSE — Root Mean Squared Error

$$\text{RMSE} = \sqrt{\text{mean}((\text{actual} - \text{predicted})^2)}$$

**In plain English:** Similar to MAE, but squares the errors first, so big mistakes are punished more. **Why it's used:** Tells you whether the model makes a few huge errors or many small ones.

### WHERE TO SHOW THIS

Same metrics table as MAE.

## 3.3 MAPE — Mean Absolute Percentage Error

$$\text{MAPE} = \text{mean}(|\text{actual} - \text{predicted}| / \text{actual}) \times 100$$

**In plain English:** The average error as a percentage of the actual price. **Why it's used:** Easy to compare across different price levels. A MAPE of 2% means the model is typically off by 2%.

### WHERE TO SHOW THIS

Same metrics table.

## 3.4 Naive Benchmark

### WHAT IT IS

The simplest possible forecast — "tomorrow's price will be today's price." Nothing fancy.

### WHY IT MATTERS

If your fancy ML model cannot beat this, it's not adding value. This is the honest baseline every model must clear.

### WHERE TO SHOW THIS

Recommendation tab > "How each model voted" table > the model with the highest MAE is usually close to the naive. Or the Forecast tab's metric tables.

## PART 4

# SARIMAX (Model 1 of 3)

**Full name:** Seasonal AutoRegressive Integrated Moving Average with eXogenous variables.

**What it is:** A classical statistical model from the 1970s. It studies the pattern of past values and past errors to predict future values.

## 4.1 The four components

| Letter | Name           | What it means                                                      |
|--------|----------------|--------------------------------------------------------------------|
| AR     | AutoRegressive | Looks at past prices (the AR order <b>p</b> )                      |
| I      | Integrated     | Differences the series to make it stationary (the order <b>d</b> ) |
| MA     | Moving Average | Looks at past forecast errors (the MA order <b>q</b> )             |
| X      | eXogenous      | External variables — ZAR/USD, rainfall, fuel                       |

### WHERE TO SHOW THIS

Forecast tab > SARIMAX > Settings expander. You see p, d, q knobs.

## 4.2 AR order (p)

“How many past days does the model look at when predicting today?”

- $p = 1$ : only yesterday matters
- $p = 3$ : the last three days matter
- Higher values = more memory, slower to fit

## 4.3 Differencing (d)

**What it is:** Subtracting each day's price from the previous day's, so you're modelling *changes* not *levels*.

**Why:** Prices tend to trend upwards over time (non-stationary). SARIMAX works best on stable series with no trend. Differencing removes the trend.

- $d = 0$ : no differencing (for already-stable data)
- $d = 1$ : standard for prices — model the daily change
- $d = 2$ : rarely needed

## 4.4 MA order (q)

“How many past forecast errors does the model correct for?” The model looks at its own recent mistakes and adjusts. Higher  $q$  = more responsive to recent errors.

## 4.5 Exogenous variables

The “X” in SARIMAX — the external drivers. The model learns a coefficient for each:

- **ZAR\_USD** coefficient = +8 = a 1-unit rand weakening is associated with an 8 R/ton price increase
- **Rainfall\_mm** coefficient = -1.2 = 1 mm more rain is associated with a 1.2 R/ton decrease

### WHERE TO SHOW THIS

Forecast tab > SARIMAX > Run > the output is the full statistical summary. Scroll to find the coefficients.

## PART 5

## XGBoost (Model 2 of 3)

**Full name:** eXtreme Gradient Boosting.

**What it is:** A machine learning model built from many small decision trees. Each tree is weak on its own, but they're combined so the errors of one tree are corrected by the next.

### 5.1 Decision Trees

A tree asks a series of yes/no questions: "Is the rand above 15.5? If yes, and fuel rose yesterday, and it's June — the price goes up." XGBoost builds hundreds of these trees and lets them vote.

### 5.2 Gradient Boosting

Each new tree is trained on the errors of the previous trees. Tree 1 gets 60% right. Tree 2 focuses on what Tree 1 got wrong. Tree 3 focuses on what Tree 2 got wrong. After 200 trees, the ensemble is much stronger than any single tree.

### 5.3 Hyperparameters

These are the settings you tune before training:

| Setting          | What it controls                                                         |
|------------------|--------------------------------------------------------------------------|
| learning_rate    | How much each tree contributes. Lower = slower but more stable           |
| n_estimators     | How many trees to build                                                  |
| max_depth        | How deep each tree can grow. Deeper = more complex = risk of overfitting |
| min_child_weight | Minimum data required in a leaf                                          |
| subsample        | Fraction of rows each tree sees                                          |
| colsample_bytree | Fraction of features each tree sees                                      |
| gamma            | Minimum loss reduction to make a split                                   |
| reg_alpha        | L1 regularisation (penalty for complexity)                               |

#### WHERE TO SHOW THIS

Forecast tab > XGBoost > Settings. Only some of these are exposed — the rest are searched automatically by RandomizedSearchCV.

### 5.4 TimeSeriesSplit (Cross-Validation)

**What it is:** A way to test the model multiple times during tuning without leaking future data. The training set always precedes the validation set. **Why it matters:** Random K-fold would let the model see future rows during tuning. That's cheating.

### 5.5 Predict diff vs level

- **Level:** predict tomorrow's absolute price (R/ton)
- **Diff:** predict tomorrow's change in price

**Why "diff" is used:** Trees cannot predict values outside their training range. If maize climbs to R5,200 but the training data maxed at R4,000, a "level" tree can never predict R5,200. But if you predict the change and add it back to the last known price, the model can track an unbounded level.

**WHERE TO SHOW THIS**

Forecast tab > XGBoost > Settings > “Predict” dropdown. Explain: “We default to 'diff' because trees can't extrapolate beyond what they've seen.”

## 5.6 Feature Importance

After training, XGBoost tells you which features were most useful. This is used to explain *why* the model made a decision.

**WHERE TO SHOW THIS**

Recommendation tab > “How each model voted” table. Or the Forecast tab if you want a chart.

## PART 6

# Seq2Seq (Model 3 of 3)

**Full name:** Sequence-to-Sequence (encoder-decoder).

**What it is:** A neural network built from two LSTMs. The first (**encoder**) reads a history of prices and drivers. The second (**decoder**) writes out the next 5 days, one at a time.

## 6.1 LSTM (Long Short-Term Memory)

A type of recurrent neural network that has internal “memory cells”. LSTMs can remember patterns from 60 days ago when making today's prediction — regular neural networks can't.

## 6.2 Encoder

Reads the last 60 business days of price + driver data and compresses them into a small “state vector” — a summary of everything the encoder has learned.

## 6.3 Decoder

Starts from the encoder's state and produces the next day's forecast. Then it takes its own output and feeds it back in to produce day 2, day 3, and so on.

## 6.4 Teacher Forcing (Training)

During training, the decoder is fed the *correct* previous value, not its own prediction. This makes training stable. At inference, it must use its own predictions — a mismatch called **exposure bias**.

## 6.5 Relative Price Encoding

Instead of feeding absolute prices (R/ton), Seq2Seq uses the **log of the price, minus the mean of the encoder window**. So the model sees relative movements.

**Why:** If prices climb above the training range, absolute prices break the model. Relative prices don't.

## 6.6 Latent Size

The size of the encoder's internal memory. Higher = more capacity = risk of overfitting. Default is 32.

## 6.7 Early Stopping

Training stops automatically if the validation error stops improving for 15 consecutive epochs. Prevents overfitting.

**WHERE TO SHOW THIS**

Forecast tab > Seq2Seq > Settings. The user can control encoder length, prediction days, latent size, max epochs.

**PART 7**

# The Ensemble

*How the three models become one decision.*

## 7.1 Weighted Average

Each model gets a weight based on how accurate it was historically:

```
weight = 1 / model's MAE
```

Models with lower MAE get higher weight. A model that was twice as accurate gets roughly twice the influence.

## 7.2 45% Cap

No single model can exceed 45% of the total weight. This prevents one model from dominating if it happened to have a lucky low error.

## 7.3 Agreement Threshold (60%)

A HOLD recommendation requires at least 60% of models (2 of 3) to agree on the direction. If the models disagree, the default is **SELL** — conservative for small farmers.

## 7.4 Safety Margin (1.33333%)

The forecast price must exceed today's price by more than the holding cost **plus** 1.33333%, before a HOLD is issued. This guards against marginal cases.

## 7.5 The Decision Logic

```
if (weighted_forecast - today_price) / today_price
    > (holding_cost + safety_margin)
    AND >= 60% of models predict a rise:
    -> HOLD

elif change_pct < -1.0:
    -> SELL (price is falling)

else:
    -> SELL (no strong signal, default to safe)
```

**WHERE TO SHOW THIS**

Recommendation tab > after running, the big coloured card tells the user HOLD or SELL, and the “How each model voted” table shows each model's individual vote, weight, and MAE.

## PART 8

# Presentation Cheat Sheet

The order in which to click through the app during a demo. **Total time: 7–8 minutes.**

| #  | Time | Where to click                                     | What to say                                                                                                                                                                                                                   |
|----|------|----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | 0:00 | Home tab                                           | "This is Kotulo. Let me show you the 30-second pitch." Read the welcome paragraph.                                                                                                                                            |
| 2  | 0:30 | Click "Try the Demo"                               | "Here's the app using realistic sample data — real SAFEX prices and drivers."                                                                                                                                                 |
| 3  | 1:00 | Home > Advanced settings                           | "Train/test split is 80/20. We only ever train on the past and test on the future."                                                                                                                                           |
| 4  | 1:30 | Home > External drivers                            | "We use three drivers: the rand, rainfall, and fuel. Each one is known to move maize prices."                                                                                                                                 |
| 5  | 2:00 | Forecast tab > SARIMAX                             | "Model 1 is SARIMAX. It's a classical statistical model. Strong at short-term forecasts and easy to interpret its coefficients."                                                                                              |
| 6  | 2:45 | Forecast tab > XGBoost                             | "Model 2 is XGBoost. Machine learning built from decision trees. It learns rules like 'if the rand weakens and fuel rises, maize goes up tomorrow'."                                                                          |
| 7  | 3:30 | Forecast tab > Seq2Seq                             | "Model 3 is Seq2Seq. A neural network that reads 60 days of history and writes out the next 5 days."                                                                                                                          |
| 8  | 4:15 | Recommendation tab > click "Get my recommendation" | "Now let me run all three at once. This takes about 60 seconds."                                                                                                                                                              |
| 9  | 5:30 | Recommendation > the big SELL/HOLD card            | "Each model is weighted by its historical accuracy. The models that were more accurate in the past have more influence. If 60% of them agree the price is rising by more than my storage costs, we say HOLD. Otherwise SELL." |
| 10 | 6:30 | Recommendation > "How each model voted" table      | "Here is the transparency: you can see each model's forecast, its weight, and its error rate."                                                                                                                                |
| 11 | 7:00 | Recommendation > Download as PDF                   | "And the farmer can save this to take to the co-op."                                                                                                                                                                          |
| 12 | 7:30 | Get Help tab                                       | "If they don't understand any term, everything is explained in plain language here — in English, Afrikaans, isiXhosa, isiZulu, or Sesotho."                                                                                   |

## PART 9

## Key Talking Points

If asked...

**“Why these three models?”**

We tested five models — including LSTM and Prophet — and dropped them because they were less accurate on this data. SARIMAX, XGBoost, and Seq2Seq are the three that consistently performed best.

**“Why not just use one model?”**

Each model looks at the data differently. SARIMAX sees statistical patterns. XGBoost learns rules. Seq2Seq reads sequences. By averaging them, we get a more robust forecast and can measure how much they agree.

**“What happens if the models disagree?”**

The app defaults to SELL. Small farmers cannot afford risk, so when there is no clear signal, we recommend safety over speculation.

**“How accurate is it?”**

That depends on the data period. Look at the test MAE for each model — the accuracy is reported honestly, and we always show a naive benchmark for comparison.

**“What if prices go outside the range the model has seen?”**

XGBoost predicts the change in price, not the level, so it can track upward trends. Seq2Seq works with relative prices for the same reason. Only the naive approach would freeze at the highest price it ever saw.

**“How do farmers use this?”**

They enter their province (which sets a default storage cost), press one button, and get a SELL or HOLD recommendation. No ML knowledge required. All the technical explanation is tucked into the Get Help tab for anyone curious.

## PART 10

## Quick Glossary

| Term                | One-line definition                             |
|---------------------|-------------------------------------------------|
| MAE                 | Average absolute error in R/ton                 |
| RMSE                | Root mean squared error — punishes big mistakes |
| MAPE                | Average error as a percentage                   |
| Naive benchmark     | “Tomorrow’s price = today’s price”              |
| Look-ahead bias     | Accidentally letting the model see the future   |
| Train/test split    | Chronological split — past trains, future tests |
| Feature engineering | Creating new input columns from raw data        |

| Term                      | One-line definition                               |
|---------------------------|---------------------------------------------------|
| <b>Lagged feature</b>     | A feature shifted back N days                     |
| <b>AR (p)</b>             | How many past prices SARIMAX uses                 |
| <b>I (d)</b>              | How many times SARIMAX differences the series     |
| <b>MA (q)</b>             | How many past errors SARIMAX corrects for         |
| <b>Exogenous</b>          | External drivers (rand, rain, fuel)               |
| <b>Gradient boosting</b>  | Building many weak trees that correct each other  |
| <b>Hyperparameter</b>     | A model setting tuned before training             |
| <b>Cross-validation</b>   | Testing on multiple time slices                   |
| <b>TimeSeriesSplit</b>    | Time-aware cross-validation (no shuffling)        |
| <b>Feature importance</b> | Which inputs the tree model used most             |
| <b>LSTM</b>               | A neural network with memory                      |
| <b>Encoder-decoder</b>    | A model that reads history then writes a forecast |
| <b>Teacher forcing</b>    | Training with correct previous values             |
| <b>Exposure bias</b>      | Train/test mismatch in Seq2Seq                    |
| <b>Ensemble</b>           | Combining multiple models into one prediction     |
| <b>Weighted average</b>   | Averaging with per-model weights                  |
| <b>Safety margin</b>      | Extra buffer on top of holding cost               |